IT3030 – Platform-based Application Frameworks

PAF Group Coursework – Final Report

Smart Campus Operations Hub

Module B – Booking Management

Group: Smart Campus Group XX

Module: IT3030

Semester: Year 3 Semester 1 – 2026

Member: Akshayan (Module B Owner)

Table of Contents

1. Introduction & Objectives
2. Requirements Analysis
   2.1 Functional Requirements
   2.2 Non-Functional Requirements
3. System Architecture
   3.1 Backend Architecture (Spring Boot)
   3.2 Frontend Architecture (React)
   3.3 Database Design
4. REST API Design
   4.1 Endpoint Catalog
   4.2 HTTP Status Codes
5. Implementation Details
   5.1 Backend Implementation
   5.2 Frontend Implementation
   5.3 Data Seeding
6. Testing & Validation
   6.1 Unit Test Results (19 Tests)
   6.2 Conflict Detection Testing
   6.3 Validation Testing
7. GitHub & Version Control
   7.1 Repository Structure
   7.2 CI/CD Pipeline
8. Screenshots & Evidence
9. Team Contributions
10. Setup & Deployment Instructions

1. Introduction & Objectives

The Smart Campus Operations Hub is a full-stack web application designed to streamline campus resource management at SLIIT. This report documents Module B – Booking Management, which provides an end-to-end system for booking campus resources such as Lecture Halls, Meeting Rooms, and Equipment.

The system is built with a Spring Boot RESTful backend and a React-based single-page frontend application, strictly adhering to layered architecture principles as mandated by the IT3030 Platform-based Application Frameworks course requirements.

Key Objectives

- Implement a fully functional booking management module with CRUD operations.
- Enforce time-slot conflict prevention using optimized JPQL queries.
- Support role-based workflows (Standard User vs. Admin) with approval/rejection flow.
- Deliver a responsive React dashboard with real-time state management.
- Maintain clean separation of concerns across Controller, Service, and Repository layers.
- Achieve comprehensive test coverage with both unit and integration tests.

2. Requirements Analysis

2.1 Functional Requirements

ID	Requirement	Description
FR-01	Resource Catalogue	The system maintains an active catalogue of bookable resources (Lecture Halls, Meeting Rooms, Equipment). Each resource has a name, type, location, capacity, and status (ACTIVE / OUT_OF_SERVICE). Only ACTIVE resources can be booked.
FR-02	Booking Requests	Standard Users can request a booking for a specific resource by providing start time, end time, purpose, and expected number of attendees. New bookings are created with PENDING status.
FR-03	Conflict Prevention	The system strictly enforces that overlapping time ranges for a specific resource are rejected with a 409 CONFLICT HTTP status. The JPQL query checks all PENDING and APPROVED bookings for overlap.
FR-04	Approval Workflow	Admin users can review all pending requests and approve or reject them. Rejection requires a mandatory reason string. Approved bookings change status to APPROVED.
FR-05	Booking Dashboard	Standard Users can view and cancel only their own PENDING/APPROVED bookings. Admins have a global view of all bookings across the campus with approve/reject actions.
FR-06	Booking Cancellation	Users can cancel their own bookings. Already cancelled bookings cannot be cancelled again (400 Bad Request).

2.2 Non-Functional Requirements

ID	Requirement	Description
NFR-01	Architecture	Strict layered RESTful API using

		Spring Boot with Controller → Service → Repository pattern.
NFR-02	Data Persistence	Relational database via JPA/Hibernate with H2 file-based database for data persistence across server restarts.
NFR-03	Validation	Bean Validation (@NotNull, @NotBlank, @FutureOrPresent, @Min) with global exception handling via @RestControllerAdvice.
NFR-04	Frontend	React-based SPA using Vite build tool. No page reloads – all interactions via AJAX/axios.
NFR-05	Concurrency	Optimistic locking via @Version annotation on Booking entity to prevent concurrent modification.
NFR-06	CI/CD	GitHub Actions workflow for automated build and test on every push/PR to main branch.

3. System Architecture

3.1 Backend Architecture (Spring Boot)

The backend follows a strict 3-tier layered architecture as required by the marking rubric:

Controller Layer (BookingController, ResourceController): Handles incoming HTTP requests using @RestController. Maps endpoints to standard RESTful principles (GET, POST, PATCH, DELETE). Uses @Valid for request body validation. Returns appropriate HTTP status codes (200, 201, 204, 400, 404, 409).

Service Layer (BookingService): Orchestrates all business logic including timestamp validation, resource availability checking, conflict prevention via repository queries, and status transition management. Uses @Transactional for data integrity. Throws custom exceptions (ResourceNotFoundException, ResourceConflictException) for error scenarios.

Repository Layer (BookingRepository, ResourceRepository, UserRepository): Extends JpaRepository for standard CRUD. Contains custom JPQL queries including existsOverlappingBooking() for conflict detection and findAllWithDetails() / findByUserIdWithDetails() for optimized JOIN FETCH queries.

Exception Handling (GlobalExceptionHandler): Centralized @RestControllerAdvice that maps exceptions to HTTP responses: ResourceNotFoundException → 404, ResourceConflictException → 409, IllegalArgumentException → 400, MethodArgumentNotValidException → 400.

Architecture Flow Diagram

```
┌──────────────────────────────────────────────────────────────┐
│                    React Frontend (Vite)                     │
│  ┌──────────────────┐    ┌──────────────────────────────┐   │
│  │ BookingDashboard │──→ │ Axios HTTP Client (REST API) │   │
│  │ (User / Admin)   │    │ baseURL: localhost:8080/api  │   │
│  └──────────────────┘    └──────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
                              │
                              │ HTTP (GET/POST/PATCH/DELETE)
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                  Spring Boot Backend                         │
│  ┌──────────────────────────────────────────────────────┐   │
│  │         Controller Layer (@RestController)           │   │
│  │   BookingController  │  ResourceController           │   │
│  └──────────────────────────────────────────────────────┘   │
│                          │                                   │
│  ┌──────────────────────────────────────────────────────┐   │
│  │           Service Layer (@Service)                   │   │
│  │   BookingService (Business Logic + Validation)       │   │
│  └──────────────────────────────────────────────────────┘   │
│                          │                                   │
│  ┌──────────────────────────────────────────────────────┐   │
│  │        Repository Layer (JpaRepository)              │   │
│  │  BookingRepo  │  ResourceRepo  │  UserRepo           │   │
│  └──────────────────────────────────────────────────────┘   │
│                          │                                   │
│  ┌──────────────────────────────────────────────────────┐   │
│  │        H2 Database (File-based: ./data/campusopsDB)  │   │
│  │   Tables: bookings, resources, users                 │   │
│  └──────────────────────────────────────────────────────┘   │
```

└──┘

3.2 Frontend Architecture (React)

Component-Based Design: The UI is built using React functional components with hooks. BookingDashboard.jsx is the primary component handling both user and admin workflows.

State Management: Uses React useState and useEffect hooks for managing booking data, form state, loading states, and error handling. Role switching triggers automatic data refresh.

API Integration: All backend communication uses Axios HTTP client. The API base URL is configured as http://localhost:8080/api. CORS is enabled on the backend for development.

Dynamic Resource Dropdown: Resources are fetched from the /api/resources endpoint and displayed in a dropdown selector, filtering only ACTIVE resources.

3.3 Database Design

The application uses H2 file-based database (jdbc:h2:file:./data/campusopsDB) with JPA/Hibernate ORM. The schema is auto-generated from entity classes using spring.jpa.hibernate.ddl-auto=update.

Entity-Relationship Summary:

Entity	Fields
Booking	id, resource_id (FK), user_id (FK), startTime, endTime, purpose, expectedAttendees, status, rejectionReason, version
Resource	id, name, type, location, capacity, status
User	id, email (unique), name, role

Relationships:

- Booking → Resource: Many-to-One (@ManyToOne, @JoinColumn)
- Booking → User: Many-to-One (@ManyToOne, @JoinColumn)
- BookingStatus Enum: PENDING, APPROVED, REJECTED, CANCELLED

4. REST API Design

4.1 Endpoint Catalog

The API strictly follows RESTful conventions with proper HTTP method usage and status codes:

HTTP Method	Endpoint	Purpose	Status Codes
GET	/api/bookings/all	Retrieve all bookings (Admin)	200 OK
GET	/api/bookings/user/{id}	Retrieve user's own bookings	200 OK
GET	/api/resources	List all campus resources	200 OK
POST	/api/bookings	Create new booking request	201 Created, 400, 409
PATCH	/api/bookings/{id}/status	Approve/Reject booking (Admin)	200 OK, 400, 404
DELETE	/api/bookings/{id}	Cancel a booking	204 No Content, 404

4.2 HTTP Status Codes

- 200 OK – Successful retrieval or update
- 201 Created – Booking successfully created
- 204 No Content – Booking successfully cancelled
- 400 Bad Request – Validation failure or invalid arguments
- 404 Not Found – Resource/Booking/User not found
- 409 Conflict – Time slot overlap detected for the requested resource

Sample Request/Response

POST /api/bookings – Create Booking Request:

```
{
  "resourceId": 1,
  "userId": 1,
  "startTime": "2026-10-10T10:00:00",
  "endTime": "2026-10-10T12:00:00",
  "purpose": "IT3030 Group Meeting",
  "expectedAttendees": 25
}
```

Response (201 Created):

```
{
  "id": 1,
  "resourceId": 1,
  "resourceName": "Lecture Hall A",
  "userId": 1,
  "userName": "Standard User",
```

```
  "startTime": "2026-10-10T10:00:00",
  "endTime": "2026-10-10T12:00:00",
  "purpose": "IT3030 Group Meeting",
  "expectedAttendees": 25,
  "status": "PENDING",
  "rejectionReason": null
}
```

5. Implementation Details

5.1 Backend Implementation

Key Code Sections:

Conflict Detection Query (BookingRepository.java):

```
@Query("SELECT COUNT(b) > 0 FROM Booking b WHERE b.resource.id = :resourceId " +
       "AND b.status IN ('PENDING', 'APPROVED') " +
       "AND (b.startTime < :endTime AND b.endTime > :startTime)")
boolean existsOverlappingBooking(@Param("resourceId") Long resourceId,
                                 @Param("startTime") LocalDateTime startTime,
                                 @Param("endTime") LocalDateTime endTime);
```

Service Layer – Create Booking with Validation (BookingService.java):

```
@Transactional
public BookingResponseDto createBookingRequest(BookingRequestDto dto) {
    if (dto.getStartTime().isAfter(dto.getEndTime())) {
        throw new IllegalArgumentException("Start time must be before end time.");
    }
    Resource resource = resourceRepository.findById(dto.getResourceId())
            .orElseThrow(() -> new ResourceNotFoundException("Resource not found"));
    if (!"ACTIVE".equals(resource.getStatus())) {
        throw new IllegalArgumentException("Resource is not currently active.");
    }
    User user = userRepository.findById(dto.getUserId())
            .orElseThrow(() -> new ResourceNotFoundException("User not found"));
    // Conflict Checking
    boolean conflict = bookingRepository.existsOverlappingBooking(
            dto.getResourceId(), dto.getStartTime(), dto.getEndTime());
    if (conflict) {
        throw new ResourceConflictException(
            "The selected resource is not available during this timeframe.");
    }
    Booking booking = Booking.builder()
            .resource(resource).user(user)
            .startTime(dto.getStartTime()).endTime(dto.getEndTime())
            .purpose(dto.getPurpose()).expectedAttendees(dto.getExpectedAttendees())
            .status(BookingStatus.PENDING).build();
    return mapToDto(bookingRepository.save(booking));
}
```

Global Exception Handler (GlobalExceptionHandler.java):

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, String>> handleResourceNotFoundException(...) {
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);   // 404
    }
    @ExceptionHandler(ResourceConflictException.class)
    public ResponseEntity<Map<String, String>> handleResourceConflictException(...) {
        return new ResponseEntity<>(error, HttpStatus.CONFLICT);    // 409
    }
```

```
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidationExceptions(...) {
        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST); // 400
    }
}
```

5.2 Frontend Implementation

The React frontend is built with Vite and uses functional components with hooks. Key features of the BookingDashboard component:

- Dynamic role switching between USER and ADMIN modes via state toggle.
- Resource dropdown populated from /api/resources API – shows only ACTIVE resources.
- Booking form with datetime-local inputs for start/end time selection.
- Admin view shows all bookings with Approve/Reject action buttons.
- User view shows only own bookings with Cancel button for PENDING/APPROVED bookings.
- Rejection requires mandatory reason via browser prompt dialog.
- Error handling with user-friendly alerts for 409 Conflict, 400 Validation, and 404 Not Found.
- Real-time refresh after every booking action (create, cancel, approve, reject).

5.3 Data Seeding

The DataSeeder component (implements CommandLineRunner) automatically seeds the database on first startup with test data:

- Users: Standard User (user@example.com, role=USER), Admin User (admin@example.com, role=ADMIN)
- Resources: Lecture Hall A (100 cap, ACTIVE), Meeting Room 1 (10 cap, ACTIVE), Sony Camera A7 (1 cap, ACTIVE), Broken Projector (OUT_OF_SERVICE)

6. Testing & Validation

6.1 Unit Test Results

The project includes 19 automated tests across 2 test classes, all passing successfully. Tests use JUnit 5, Mockito, and Spring MockMvc.

Test Execution Summary:

```
-------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running com.campusops.booking.controller.BookingControllerTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.547 s

Running com.campusops.booking.service.BookingServiceTest
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.331 s

Results:
Tests run: 19, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS
```

BookingServiceTest – 13 Unit Tests:

#	Test Name	Scenario	Result
1	getAllBookings_ShouldReturnAllBookings	Verifies all bookings are returned with details	**PASS**
2	getUserBookings_ShouldReturnUserBookings	Verifies user-specific booking retrieval	**PASS**
3	createBookingRequest_Success	Creates booking with valid data, returns PENDING	**PASS**
4	createBookingRequest_InvalidTimeRange	Rejects when start > end time	**PASS**
5	createBookingRequest_ResourceNotFound	Throws 404 for non-existent resource	**PASS**
6	createBookingRequest_ConflictDetected	Throws 409 for overlapping time slot	**PASS**
7	createBookingRequest_InactiveResource	Rejects booking on OUT_OF_SERVICE resource	**PASS**
8	updateStatus_ApproveBooking	Admin approves a PENDING booking	**PASS**
9	updateStatus_RejectWithReason	Admin rejects with	**PASS**

		mandatory reason	
10	updateStatus_RejectWithoutReason	Throws error when rejection reason is empty	**PASS**
11	cancelBooking_Success	Cancels a valid booking	**PASS**
12	cancelBooking_AlreadyCancelled	Prevents double cancellation	**PASS**
13	cancelBooking_NotFound	Throws 404 for non-existent booking	**PASS**

BookingControllerTest – 6 Integration Tests:

#	Test Name	Scenario	Result
1	getAllBookings_Returns200	GET /api/bookings/all returns 200 + JSON	**PASS**
2	getUserBookings_Returns200	GET /api/bookings/user/1 returns 200	**PASS**
3	createBooking_Returns201	POST /api/bookings returns 201 Created	**PASS**
4	createBooking_Returns400_InvalidRequest	POST with missing fields returns 400	**PASS**
5	updateBookingStatus_Returns200	PATCH /api/bookings/1/status returns 200	**PASS**
6	cancelBooking_Returns204	DELETE /api/bookings/1 returns 204 No Content	**PASS**

6.2 Conflict Detection Testing

The conflict prevention mechanism was verified with the following scenario:

- Step 1: User books "Lecture Hall A" from 10:00 AM to 12:00 PM on 10/10/2026 → 201 Created (PENDING)
- Step 2: Another user attempts to book same resource from 11:00 AM to 1:00 PM → 409 CONFLICT
- Step 3: The JPQL query SELECT COUNT(b) > 0 FROM Booking b WHERE ... detects the overlap
- Step 4: ResourceConflictException is thrown and mapped to HTTP 409 via @RestControllerAdvice
- Step 5: React UI displays alert: "Conflict: Resource is already booked for this time."

6.3 Validation Testing

Bean Validation constraints tested across the following scenarios:

Scenario	Validation	HTTP Response
Missing resourceId	@NotNull	400 Bad Request
Blank purpose	@NotBlank	400 Bad Request

Past start time	@FutureOrPresent	400 Bad Request
Attendees = 0	@Min(1)	400 Bad Request
Start after end	Service logic	400 Bad Request
Rejection without reason	Service logic	400 Bad Request
Inactive resource	Service logic	400 Bad Request

7. GitHub & Version Control

7.1 Repository Structure

```
it3030-paf-2026-smart-campus-groupXX/
├── .github/
│   └── workflows/
│       └── ci.yml                    # GitHub Actions CI pipeline
├── .gitignore
├── README.md
├── backend/
│   ├── pom.xml
│   ├── mvnw.cmd
│   └── src/
│       ├── main/java/com/campusops/booking/
│       │   ├── BookingApplication.java
│       │   ├── DataSeeder.java
│       │   ├── controller/
│       │   │   ├── BookingController.java
│       │   │   └── ResourceController.java
│       │   ├── dto/
│       │   │   ├── BookingRequestDto.java
│       │   │   ├── BookingResponseDto.java
│       │   │   └── BookingStatusUpdateDto.java
│       │   ├── exception/
│       │   │   ├── GlobalExceptionHandler.java
│       │   │   ├── ResourceConflictException.java
│       │   │   └── ResourceNotFoundException.java
│       │   ├── model/
│       │   │   ├── Booking.java
│       │   │   ├── BookingStatus.java
│       │   │   ├── Resource.java
│       │   │   └── User.java
│       │   ├── repository/
│       │   │   ├── BookingRepository.java
│       │   │   ├── ResourceRepository.java
│       │   │   └── UserRepository.java
│       │   └── service/
│       │       └── BookingService.java
│       ├── main/resources/
│       │   └── application.properties
│       └── test/java/com/campusops/booking/
│           ├── controller/
│           │   └── BookingControllerTest.java
│           └── service/
│               └── BookingServiceTest.java
├── docs/
│   ├── IT3030_PAF_Assignment_2026_GroupXX.md
│   └── IT3030_PAF_Report.docx
└── frontend/
    ├── package.json
    ├── vite.config.js
    ├── index.html
    └── src/
        ├── App.jsx
        ├── main.jsx
        ├── index.css
        └── components/
```

```
└── BookingDashboard.jsx
```

7.2 CI/CD Pipeline (GitHub Actions)

A GitHub Actions workflow (.github/workflows/ci.yml) is configured to run on every push and pull request to the main branch. It performs the following steps:

- Backend Job: Checkout → Set up JDK 17 → Cache Maven → Run mvn clean verify (compile + test)
- Frontend Job: Checkout → Set up Node.js 20 → npm install → npm run build

CI Workflow YAML:

```yaml
name: Smart Campus CI
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  backend-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin' }
      - run: cd backend && mvn clean verify
  frontend-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: cd frontend && npm install && npm run build
```

8. Screenshots & Evidence

This section presents captured screenshots from the running application, API testing terminal outputs, automated test results, and version control evidence.

8.1 User Interface

User Dashboard – Booking Form

Shows the Standard User view with the resource dropdown, date/time pickers, purpose field, and attendees input. The "Submit Request" button is visible.

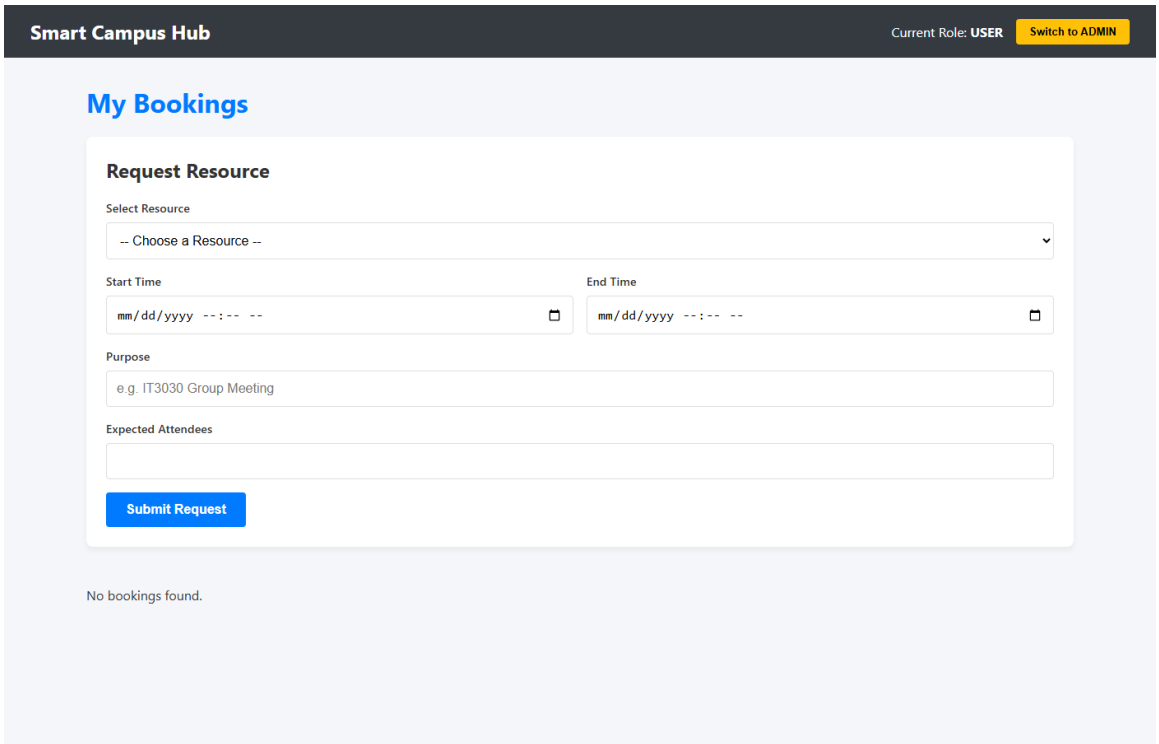

Figure 8.1: User Dashboard – Empty Booking Form

User Dashboard – Filled Booking Form

Demonstrates the form filled with booking details before submission.

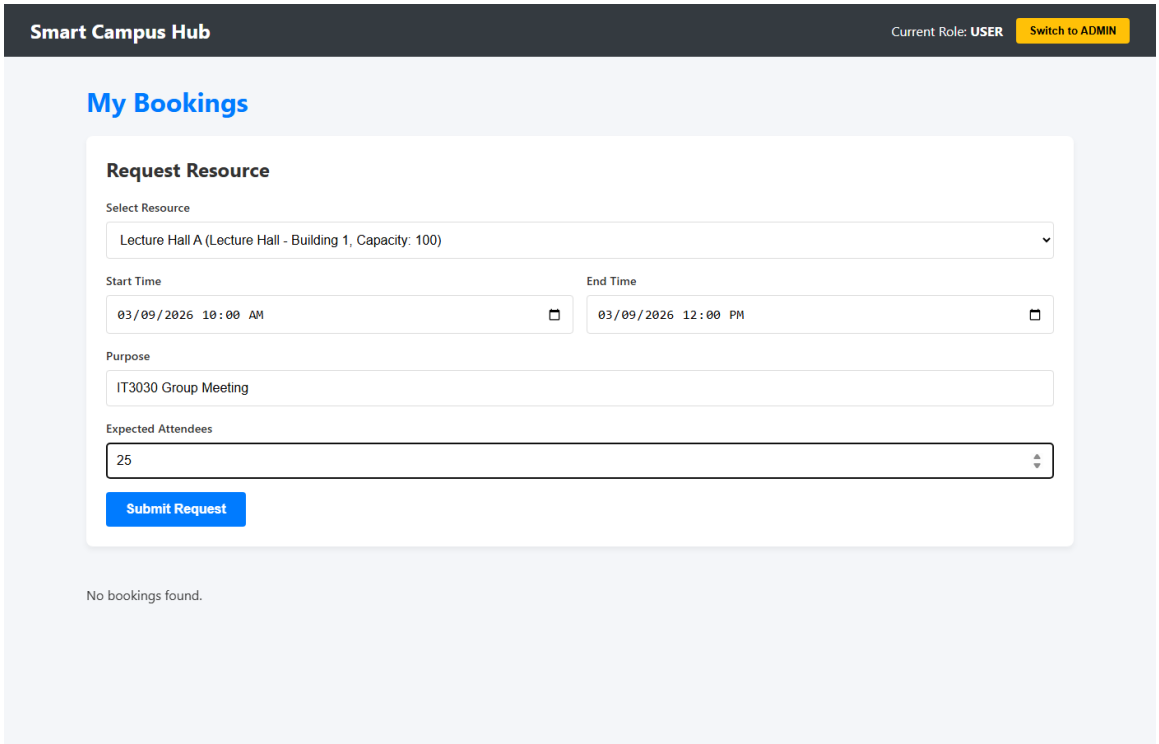

Figure 8.2: User Dashboard – Filled Booking Form

User Dashboard – Booking Cards (Pending)

Displays booked resources as cards with PENDING status badges (yellow). Cancel button is visible on each booking card.

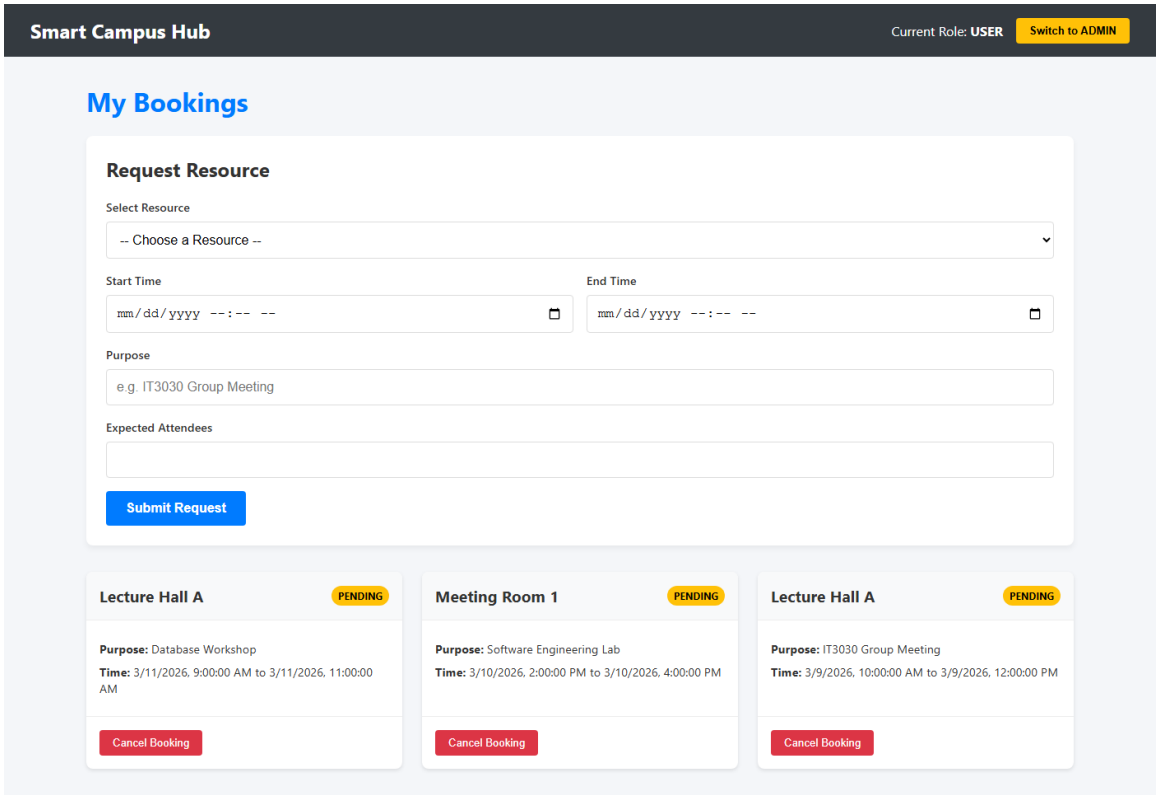

Figure 8.3: User Dashboard – Pending Booking Cards

Admin Dashboard – All Pending Bookings

Shows the Admin view listing all bookings across the campus. Each pending booking shows Approve (green) and Reject (red) action buttons.

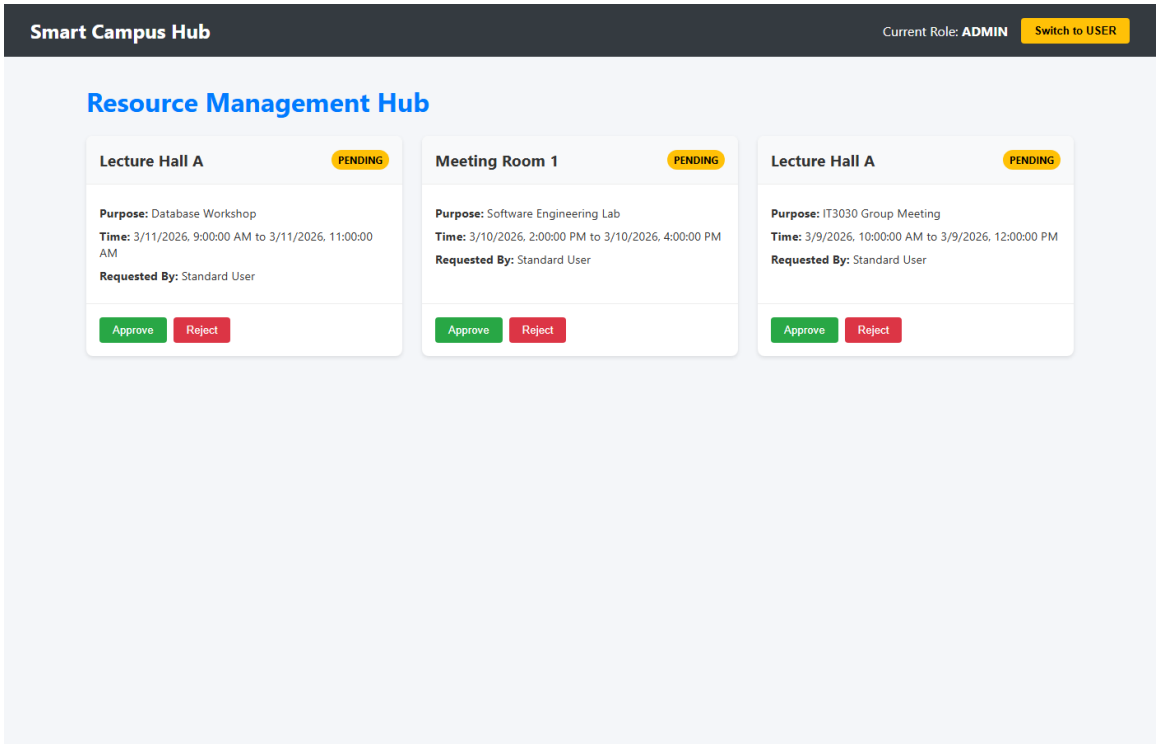

Figure 8.4: Admin Dashboard – All Pending Bookings

Admin Dashboard – After Approve/Reject

Shows the Admin view after approving and rejecting bookings. Status badges reflect APPROVED (green) and REJECTED (red) states with rejection reason.

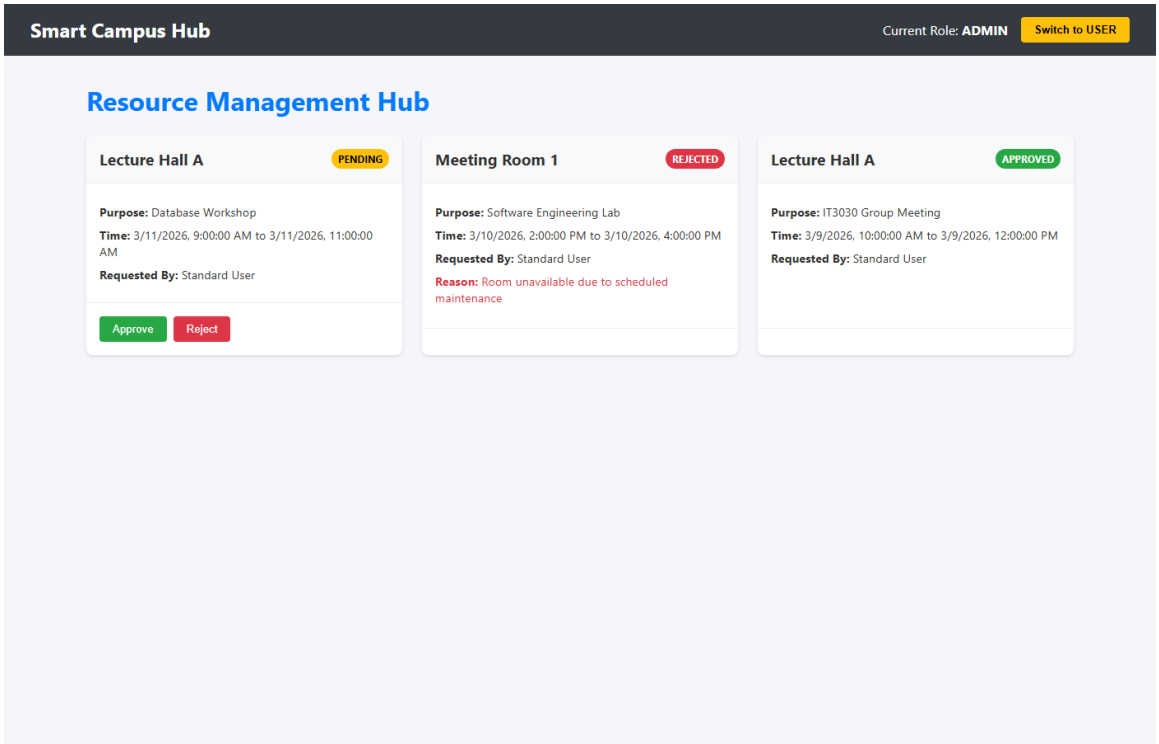

Figure 8.5: Admin Dashboard – After Approve/Reject Actions

User View – Mixed Booking Statuses

User view showing bookings with mixed statuses: APPROVED (green), REJECTED (red with reason), and PENDING (yellow).

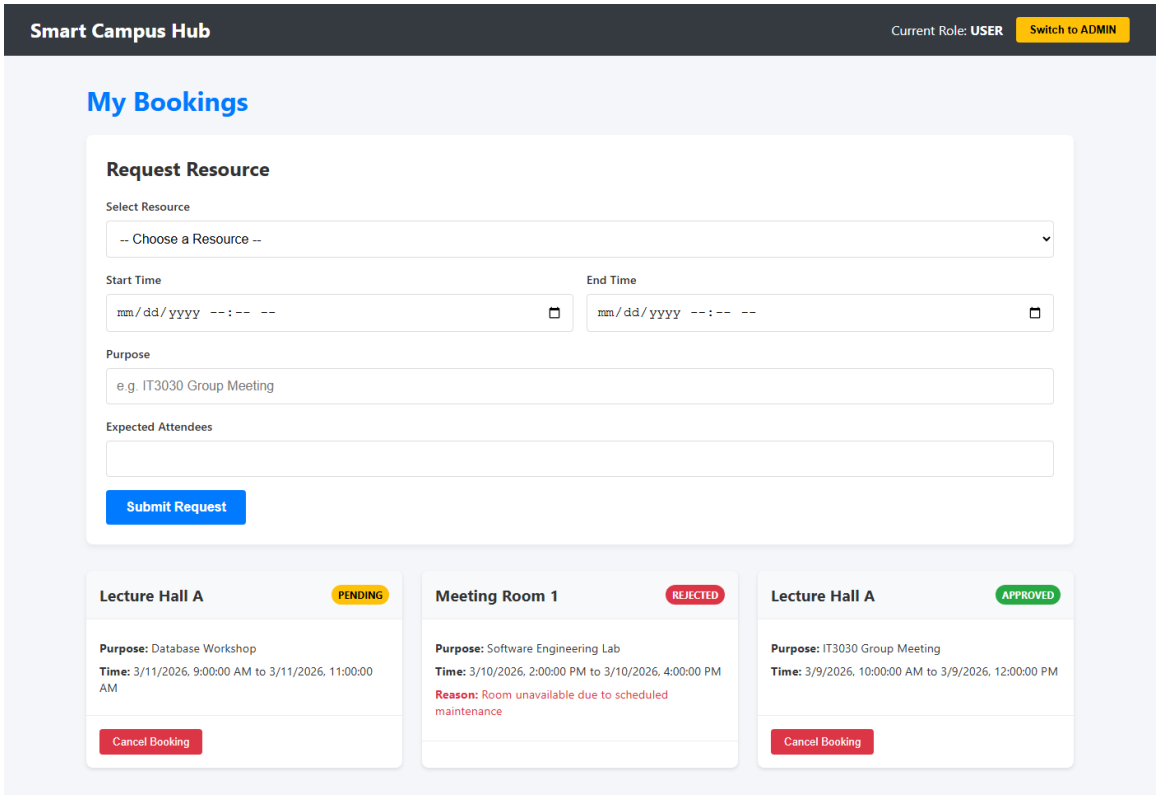

Figure 8.6: User View – Mixed Booking Statuses

8.2 API Testing (Terminal Outputs)

The following screenshots show REST API testing performed via terminal commands. Each screenshot demonstrates the HTTP request, response status, and JSON response body.

GET All Resources

Retrieves all available campus resources (200 OK).

```
API Testing - GET All Resources

PS C:\> Invoke-RestMethod -Uri 'http://localhost:8080/api/resources' -Method GET
HTTP Status: 200
[
  {
    "id": 1,
    "name": "Lecture Hall A",
    "type": "Lecture Hall",
    "location": "Building 1",
    "capacity": 100,
    "status": "ACTIVE"
  },
  {
    "id": 2,
    "name": "Meeting Room 1",
    "type": "Meeting Room",
    "location": "Library",
    "capacity": 10,
    "status": "ACTIVE"
  },
  {
    "id": 3,
    "name": "Sony Camera A7",
    "type": "Equipment",
    "location": "Media Lab",
    "capacity": 1,
    "status": "ACTIVE"
  },
  {
    "id": 4,
    "name": "Broken Projector",
    "type": "Equipment",
    "location": "Store",
    "capacity": 1,
    "status": "OUT_OF_SERVICE"
  }
]
```

Figure 8.7: GET /api/resources – 200 OK

POST Create Booking (201 Created)

Successfully creates a new booking request with valid data.

```
API Testing - POST Create Booking (201 Created)

PS C:\> Invoke-RestMethod -Uri 'http://localhost:8080/api/bookings' -Method POST -ContentType 'application/json' -Body '{"resourceId": 3,
"userId": 1, "startTime": "2026-03-12T15:00:00", "endTime": "2026-03-12T17:00:00", "purpose": "Photography Session", "expectedAttendees": 1}'
HTTP Status: 201
{
  "id": 4,
  "resourceId": 3,
  "resourceName": "Sony Camera A7",
  "userId": 1,
  "userName": "Standard User",
  "startTime": "2026-03-12T15:00:00",
  "endTime": "2026-03-12T17:00:00",
  "purpose": "Photography Session",
  "expectedAttendees": 1,
  "status": "PENDING",
  "rejectionReason": null
}
```

Figure 8.8: POST /api/bookings – 201 Created

GET User Bookings

Retrieves all bookings for a specific user by ID (200 OK).

API Testing - GET User Bookings (200 OK)

```
PS C:\> Invoke-RestMethod -Uri 'http://localhost:8080/api/bookings/user/1' -Method GET
HTTP Status: 200
[
  {
    "id": 4,
    "resourceId": 3,
    "resourceName": "Sony Camera A7",
    "userId": 1,
    "userName": "Standard User",
    "startTime": "2026-03-12T15:00:00",
    "endTime": "2026-03-12T17:00:00",
    "purpose": "Photography Session",
    "expectedAttendees": 1,
    "status": "PENDING",
    "rejectionReason": null
  },
  {
    "id": 3,
    "resourceId": 1,
    "resourceName": "Lecture Hall A",
    "userId": 1,
    "userName": "Standard User",
    "startTime": "2026-03-11T09:00:00",
    "endTime": "2026-03-11T11:00:00",
    "purpose": "Database Workshop",
    "expectedAttendees": 40,
    "status": "PENDING",
    "rejectionReason": null
  },
  {
    "id": 2,
    "resourceId": 2,
    "resourceName": "Meeting Room 1",
    "userId": 1,
    "userName": "Standard User",
    "startTime": "2026-03-10T14:00:00",
    "endTime": "2026-03-10T16:00:00",
    "purpose": "Software Engineering Lab",
    "expectedAttendees": 10,
    "status": "REJECTED",
    "rejectionReason": "Room unavailable due to scheduled maintenance"
  },
  {
    "id": 1,
    "resourceId": 1,
    "resourceName": "Lecture Hall A",
    "userId": 1,
    "userName": "Standard User",
    "startTime": "2026-03-09T10:00:00",
    "endTime": "2026-03-09T12:00:00",
    "purpose": "IT3030 Group Meeting",
    "expectedAttendees": 25,
    "status": "APPROVED",
    "rejectionReason": null
  }
]
```

Figure 8.9: GET /api/bookings/user/1 – 200 OK

GET All Bookings (Admin)

Admin endpoint retrieves all bookings across the system (200 OK).

API Testing - GET All Bookings / Admin (200 OK)

```
PS C:\> Invoke-RestMethod -Uri 'http://localhost:8080/api/bookings/all' -Method GET
HTTP Status: 200
[
  {
    "id": 4,
    "resourceId": 3,
    "resourceName": "Sony Camera A7",
    "userId": 1,
    "userName": "Standard User",
    "startTime": "2026-03-12T15:00:00",
    "endTime": "2026-03-12T17:00:00",
    "purpose": "Photography Session",
    "expectedAttendees": 1,
    "status": "PENDING",
    "rejectionReason": null
  },
  {
    "id": 3,
    "resourceId": 1,
    "resourceName": "Lecture Hall A",
    "userId": 1,
    "userName": "Standard User",
    "startTime": "2026-03-11T09:00:00",
    "endTime": "2026-03-11T11:00:00",
    "purpose": "Database Workshop",
    "expectedAttendees": 40,
    "status": "PENDING",
    "rejectionReason": null
  },
  {
    "id": 2,
    "resourceId": 2,
    "resourceName": "Meeting Room 1",
    "userId": 1,
    "userName": "Standard User",
    "startTime": "2026-03-10T14:00:00",
    "endTime": "2026-03-10T16:00:00",
    "purpose": "Software Engineering Lab",
    "expectedAttendees": 10,
    "status": "REJECTED",
    "rejectionReason": "Room unavailable due to scheduled maintenance"
  },
  {
    "id": 1,
    "resourceId": 1,
    "resourceName": "Lecture Hall A",
    "userId": 1,
    "userName": "Standard User",
    "startTime": "2026-03-09T10:00:00",
    "endTime": "2026-03-09T12:00:00",
    "purpose": "IT3030 Group Meeting",
    "expectedAttendees": 25,
    "status": "APPROVED",
    "rejectionReason": null
  }
]
```

Figure 8.10: GET /api/bookings/all – 200 OK

PATCH Approve Booking

Admin approves a pending booking request (200 OK).

```
API Testing - PATCH Approve Booking (200 OK)

PS C:\> Invoke-RestMethod -Uri 'http://localhost:8080/api/bookings/4/status' -Method PATCH -Body '{"status":"APPROVED"}'
HTTP Status: 200
{
  "id": 4,
  "resourceId": 3,
  "resourceName": "Sony Camera A7",
  "userId": 1,
  "userName": "Standard User",
  "startTime": "2026-03-12T15:00:00",
  "endTime": "2026-03-12T17:00:00",
  "purpose": "Photography Session",
  "expectedAttendees": 1,
  "status": "APPROVED",
  "rejectionReason": null
}
```

Figure 8.11: PATCH /api/bookings/{id}/status – Approve – 200 OK

POST Conflict Detection (409)

Demonstrates the 409 Conflict response when attempting to book a resource that has an overlapping approved/pending booking for the same time slot.

```
API Testing - POST Conflict Detection (409 Conflict)

PS C:\> Invoke-RestMethod -Uri 'http://localhost:8080/api/bookings' -Method POST -Body '{"resourceId":1,"startTime":"2026-03-
09T10:00:00","endTime":"2026-03-09T12:00:00",...}'
HTTP Status: 409
{
  "error": "The selected resource is not available during this timeframe."
}
```

Figure 8.12: POST /api/bookings – 409 Conflict

POST Validation Error (400)

Demonstrates the 400 Bad Request response for invalid/missing booking data.

```
API Testing - POST Validation Error (400 Bad Request)

PS C:\> Invoke-RestMethod -Uri 'http://localhost:8080/api/bookings' -Method POST -Body '{}'
HTTP Status: 400
{
  "timestamp": "2026-03-08T12:31:31.378+00:00",
  "status": 400,
  "error": "Bad Request",
  "path": "/api/bookings"
}
```

Figure 8.13: POST /api/bookings – 400 Bad Request

DELETE Cancel Booking (204)

User cancels an existing booking (204 No Content).

```
API Testing - DELETE Cancel Booking (204 No Content)

PS C:\> Invoke-RestMethod -Uri 'http://localhost:8080/api/bookings/4' -Method DELETE
HTTP Status: 204
(empty response body - 204 No Content)
```

Figure 8.14: DELETE /api/bookings/{id} – 204 No Content

8.3 Automated Test Results

Maven test execution output showing all 19 unit tests passing successfully (13 service tests + 6 controller tests). Tests cover CRUD operations, conflict detection, approval workflow, and error handling.

```
Maven Test Results - Module B (Booking Management)

[INFO] -----------------------< com.campusops:booking >------------------------
[INFO] Building booking 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ booking ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ booking ---
[INFO] Changes detected - recompiling the module! :input tree
[INFO] Compiling 18 source files with javac [debug release 17] to target\classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ booking ---
[INFO] skip non existing resourceDirectory c:\Users\akshayan\Documents\sliit\PAF\assignment\it3030-paf-2026-smart-campus-groupXX\backend\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ booking ---
[INFO] Changes detected - recompiling the module! :dependency
[INFO] Compiling 2 source files with javac [debug release 17] to target\test-classes
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ booking ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.campusops.booking.controller.BookingControllerTest
18:01:39.962 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils -- Could not detect default configuration classes for test
class [com.campusops.booking.controller.BookingControllerTest]: BookingControllerTest does not declare any static, non-private, non-final, nested classes
annotated with @Configuration.
18:01:40.120 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper -- Found @SpringBootConfiguration
com.campusops.booking.BookingApplication for test class com.campusops.booking.controller.BookingControllerTest

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.2.4)

2026-03-08T18:01:40.507+05:30  INFO 45068 --- [booking] [           main] c.c.b.controller.BookingControllerTest   : Starting BookingControllerTest using Java
17.0.12 with PID 45068 (started by akshayan in c:\Users\akshayan\Documents\sliit\PAF\assignment\it3030-paf-2026-smart-campus-groupXX\backend)
2026-03-08T18:01:40.509+05:30  INFO 45068 --- [booking] [           main] c.c.b.controller.BookingControllerTest   : No active profile set, falling back to 1
default profile: "default"
2026-03-08T18:01:43.090+05:30  INFO 45068 --- [booking] [           main] o.s.b.t.m.w.SpringBootMockServletContext : Initializing Spring TestDispatcherServlet
''
2026-03-08T18:01:43.091+05:30  INFO 45068 --- [booking] [           main] o.s.t.web.servlet.TestDispatcherServlet  : Initializing Servlet ''
2026-03-08T18:01:43.098+05:30  INFO 45068 --- [booking] [           main] o.s.t.web.servlet.TestDispatcherServlet  : Completed initialization in 4 ms
2026-03-08T18:01:43.156+05:30  INFO 45068 --- [booking] [           main] c.c.b.controller.BookingControllerTest   : Started BookingControllerTest in 2.949
seconds (process running for 3.9)
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.869 s -- in com.campusops.booking.controller.BookingControllerTest
[INFO] Running com.campusops.booking.service.BookingServiceTest
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.391 s -- in com.campusops.booking.service.BookingServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] Total time:  10.101 s
[INFO] Finished at: 2026-03-08T18:01:44+05:30
```

Figure 8.15: Maven Test Results – 19 Tests Passed, BUILD SUCCESS

8.4 Version Control & GitHub

Git commit history showing structured conventional commits and the remote GitHub repository configuration.

GitHub Repository - Version Control

Remote:

```
origin  https://github.com/AK29-Shay/PAF-member2.git (fetch)
origin  https://github.com/AK29-Shay/PAF-member2.git (push)
```

Commit History:

```
* ea8d7d8 (HEAD -> main, origin/main, origin/HEAD) chore: add Maven wrapper for portable builds
* 5e66eb7 docs: add project documentation and assignment report
* 6f1dec3 ci: add GitHub Actions workflow for backend and frontend builds
* bb65d94 feat(frontend): add React booking dashboard with Vite
* dfb5e76 test(backend): add unit and integration tests for booking module
* ef78b56 feat(backend): add Spring Boot booking management module
* 2992d24 chore: add .gitignore for Java, Node, IDE, and OS files
* 3d13bf6 Initial commit
```

Repository URL:

https://github.com/AK29-Shay/PAF-member2

Figure 8.16: Git Commit History & GitHub Remote

9. Team Contributions

This section documents individual member contributions to the project. Module B (Booking Management) was developed entirely by Akshayan.

Member	Module	Contributions
Member 1	Module A	(To be filled by team member)
Akshayan (Member 2)	Module B Booking Management	• Booking entity model with @ManyToOne relationships and @Version optimistic lock • BookingService with full CRUD, conflict detection, and approval workflow • BookingController REST API (GET, POST, PATCH, DELETE) • ResourceController for resource listing API • GlobalExceptionHandler for centralized error handling • Custom DTOs (BookingRequestDto, BookingResponseDto, BookingStatusUpdateDto) • DataSeeder for test data initialization • BookingServiceTest (13 unit tests) and BookingControllerTest (6 integration tests) • React BookingDashboard with user/admin role switching • GitHub Actions CI pipeline • Full project documentation and report
Member 3	Module C	(To be filled by team member)
Member 4	Module D	(To be filled by team member)

10. Setup & Deployment Instructions

Prerequisites

- Java 17+ (JDK)
- Node.js 18+ and npm
- Maven 3.9+ (or use included mvnw wrapper)
- Git

10.1 Backend Setup

1. Navigate to the backend directory: cd backend

2. Run the application: mvnw.cmd spring-boot:run (Windows) or ./mvnw spring-boot:run (Linux/Mac)

3. The server starts on http://localhost:8080

4. H2 Console available at http://localhost:8080/h2-console (JDBC URL: jdbc:h2:file:./data/campusopsDB)

5. Database is auto-seeded with test users and resources on first startup

6. Run tests: mvnw.cmd test

10.2 Frontend Setup

1. Navigate to the frontend directory: cd frontend

2. Install dependencies: npm install

3. Start the dev server: npm run dev

4. Access the UI at http://localhost:5173

5. Use the "Switch to ADMIN/USER" button to toggle between roles

6. Build for production: npm run build

10.3 API Testing

The API can be tested using curl, Postman, or any HTTP client. Ensure the backend is running on port 8080 before starting the frontend.

— End of Report —